Exercise 8

Joshua John E. Santiago

2026-11-06

#R Lab: Basic Operations - sort(), rank(), and order()

#Part A — Creating Vectors and Sorting

#Question #1

```
# Part a: create the vector
scores <- c(Ana = 78, Ben = 92, Cid = 85, Dex = 92, Eli = 67, Fay = 88, Gio = 74)
scores
```

```
## Ana Ben Cid Dex Eli Fay Gio
##  78  92  85  92  67  88  74
```

```
# Part b: sort ascending and descending
sort(scores)
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
sort(scores, decreasing = TRUE)
```

```
## Ben Dex Fay Cid Ana Gio Eli
##  92  92  88  85  78  74  67
```

```
# Part c: check that the names stayed with their values
sort_asc <- sort(scores)
print(sort_asc)
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Part d: create the other vectors
subjects <- c("Science", "Math", "English", "Filipino", "PE")
incomplete <- c(85, NA, 90, 70, NA, 95)
```

```
# Part e: sort subjects alphabetically
sort(subjects)
```

```
## [1] "English"  "Filipino" "Math"     "PE"       "Science"
```

```
# Part f: sort incomplete, NA values are removed by default
sort(incomplete)
```

```
## [1] 70 85 90 95
```

```
# Now sort again with na.last = TRUE so the NAs appear at the end
sort(incomplete, na.last = TRUE)
```

```
## [1] 70 85 90 95 NA NA
```

#Question #2

```
# Answer: Yes, the names stayed attached to their original values. For example,
# Eli is still 67 and Dex is still 92. This matters because if the names did not
# follow the values, I could pick the wrong top performer. The highest score
# would be matched to the wrong student.
```

#Question #3

```
# Answer: sort() drops the NA values without any warning. If I report the lowest
# score from the sorted result, I would not know that some data was missing.
# The real lowest score could be one of the missing values, and I would also
# be counting fewer people than I really have. So I should check for NA first.
```

#Question #4

```
order(scores)
```

```
## [1] 5 7 1 3 6 2 4
```

```
sort(scores)
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Answer: sort(scores) gives the values themselves arranged from lowest to
# highest. order(scores) gives the positions (5 7 1 3 6 2 4), which tell which
# element goes first, second, and so on.
```

```
# Part a: use order() to index scores
scores[order(scores)]
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
sort(scores)
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Answer: Yes, they have the same values. order(scores) gives the positions
# in sorted order, and putting those positions inside scores[ ] picks out the
# values in that order, which is the same as what sort() does.
```

#Question #5

```
students <- data.frame(
  name = c("Ana", "Ben", "Cid", "Dex", "Eli", "Fay", "Gio"),
  score = c(78, 92, 85, 92, 67, 88, 74),
  section = c("A", "B", "A", "B", "A", "B", "A")
)
print(students)
```

```
##   name score section
## 1  Ana    78       A
## 2  Ben    92       B
## 3  Cid    85       A
## 4  Dex    92       B
## 5  Eli    67       A
## 6  Fay    88       B
## 7  Gio    74       A
```

```r
# Part a: reorder the entire data frame by score
students[order(students$score), ]
```

```
##   name score section
## 5  Eli    67       A
## 7  Gio    74       A
## 1  Ana    78       A
## 3  Cid    85       A
## 6  Fay    88       B
## 2  Ben    92       B
## 4  Dex    92       B
```

```r
# Answer: The whole rows are arranged from the lowest score (Eli, 67) to the
# highest (Ben and Dex, 92), and the name and section move with each score.
```

```r
# Part b: reorder by score, descending
students[order(students$score, decreasing = TRUE), ]
```

```
##   name score section
## 2  Ben    92       B
## 4  Dex    92       B
## 6  Fay    88       B
## 3  Cid    85       A
## 1  Ana    78       A
## 7  Gio    74       A
## 5  Eli    67       A
```

#Question #6

```r
order(scores)
```

```
## [1] 5 7 1 3 6 2 4
```

```r
# Answer: order(scores) returns 5 7 1 3 6 2 4. These are not scores, they are
# the positions of the values, arranged from the smallest value to the largest.
# By itself it is useless because it does not show any actual values or names.
# It only becomes useful when I put it inside brackets to index something, like
# scores[order(scores)].
```

#Question #7

```r
# Answer: sort() only works on a single vector, so it can not sort a table
# with many columns. If I ran sort(students), it would not know which column to
# use, and it would not keep the rows together. order() is the correct tool
# because it gives the row positions, and I can use those positions to reorder
# every column at once, so the name, score, and section stay lined up.
```

#Question #8

```r
# Use rank() and print the result
rank(scores)
```

```
## Ana Ben Cid Dex Eli Fay Gio
## 3.0 6.5 4.0 6.5 1.0 5.0 2.0
```

```r
# Scores and ranks side by side
cbind(scores, rank(scores))
```

```
##     scores
```

```
## Ana     78 3.0
## Ben     92 6.5
## Cid     85 4.0
## Dex     92 6.5
## Eli     67 1.0
## Fay     88 5.0
## Gio     74 2.0
```

```
# Part a: look at the rank for Ben and Dex
rank(scores)[c("Ben", "Dex")]
```

```
## Ben Dex
## 6.5 6.5
```

```
# Answer: Ben and Dex both received a rank of 6.5.
```

```
# Part b: Answer: Ben and Dex are tied at 92, and they would be rank 6 and
# rank 7 from the lowest. By default rank() gives tied values the average of
# the ranks they share, which is (6 + 7) / 2 = 6.5. R does this so that
# neither of them is favored over the other and the ranks still add up
# correctly.
```

#Question #8 (second part)

```
# Part a: create the data frame and add the rank column
sales <- data.frame(
  employee = c("Rey", "Sam", "Tin", "Uma", "Val"),
  units_sold = c(120, 145, 145, 98, 130)
)

# The top seller gets rank 1, so I rank the negative values
sales$rank <- rank(-sales$units_sold)
print(sales)
```

```
##   employee units_sold rank
## 1      Rey        120  4.0
## 2      Sam        145  1.5
## 3      Tin        145  1.5
## 4      Uma         98  5.0
## 5      Val        130  3.0
```

```
# Part b: reorder the whole data frame from highest to lowest units_sold
sales[order(sales$units_sold, decreasing = TRUE), ]
```

```
##   employee units_sold rank
## 2      Sam        145  1.5
## 3      Tin        145  1.5
## 5      Val        130  3.0
## 1      Rey        120  4.0
## 4      Uma         98  5.0
```

#Question #9

```
# Answer: No, I could not have used order() alone. order() can arrange the
# rows, but it only gives positions, so it does not tell the standing of each
# employee. For example, it would not show that Sam and Tin are tied. rank()
# gives each value its standing as a number and handles ties, while order()
# only tells where each row should go in the display.
```